1. INTITULÉ DU PROJET

Nom du projet : BizFlow

Thème : Conception et développement d'une plateforme web de gestion intégrée pour petites entreprises

Type de stage : Stage freelance académique

Durée : 3 mois

Stack technique : Next.js · TypeScript · PostgreSQL · TailwindCSS

BizFlow est une plateforme web permettant à une petite entreprise de centraliser la gestion de ses clients, produits et services, commandes, paiements, utilisateurs et activités à travers une interface web moderne et sécurisée.

2. CONTEXTE ET JUSTIFICATION

De nombreuses petites entreprises utilisent encore plusieurs outils distincts pour gérer leurs activités quotidiennes : fichiers Excel, cahiers, applications de messagerie ou documents séparés. Cette organisation fragmentée engendre plusieurs problèmes :

- Des données dispersées et difficiles à retrouver
- Des erreurs fréquentes de saisie
- Des difficultés de suivi des clients et des commandes
- Une mauvaise visibilité sur les ventes et les paiements
- L'absence d'un historique centralisé et traçable
- L'impossibilité de produire des statistiques fiables

Le projet BizFlow vise à répondre à ces besoins en proposant une solution unique, centralisée et accessible via un navigateur web.

3. PROBLÉMATIQUE

Comment concevoir et développer une application web permettant à une petite entreprise de centraliser et de gérer efficacement ses clients, produits et services, commandes et paiements, tout en assurant la sécurité, l'intégrité des données et une expérience utilisateur moderne ?

4. OBJECTIFS

4.1 Objectif général

L'objectif général est de concevoir, développer, tester et déployer une application web full-stack de gestion d'entreprise. L'application devra permettre aux utilisateurs autorisés de :

- Gérer les clients
- Gérer les produits et services
- Gérer les commandes
- Suivre les paiements
- Consulter un tableau de bord analytique
- Gérer les utilisateurs et les rôles
- Consulter l'historique des opérations

4.2 Objectifs spécifiques

Sur le plan technique, le projet devra permettre de :

- Concevoir une interface web responsive avec HTML, CSS et TailwindCSS
- Utiliser TypeScript pour sécuriser le développement
- Développer l'interface avec React et Next.js
- Concevoir et administrer une base de données PostgreSQL
- Développer une API REST pour la communication frontend/backend
- Mettre en place un système d'authentification sécurisé
- Implémenter une gestion des rôles et des permissions
- Réaliser les opérations CRUD complètes
- Ajouter des validations côté client et côté serveur
- Implémenter des tests unitaires et fonctionnels
- Conteneuriser l'application avec Docker
- Déployer l'application en ligne

5. UTILISATEURS DU SYSTÈME

L'application comportera trois types d'utilisateurs avec des niveaux d'accès différenciés.

Administrateur

- Gérer les utilisateurs et les rôles
- Consulter toutes les données du système
- Gérer les paramètres généraux de l'application
- Consulter les statistiques globales
- Consulter l'historique de toutes les opérations

Gestionnaire

- Gérer les clients
- Gérer les produits et services
- Créer et modifier les commandes
- Enregistrer les paiements
- Consulter les statistiques liées à son activité

Utilisateur / Employé

- Consulter les clients et les produits
- Créer certaines opérations selon les permissions attribuées
- Consulter ses propres activités

Les permissions de chaque utilisateur sont déterminées exclusivement par le rôle qui lui est attribué par l'administrateur.

6. MODULES FONCTIONNELS

6.1 Module Authentification

Ce module constitue le point d'entrée de l'application. Il permettra à l'utilisateur de :

- Se connecter avec ses identifiants
- Se déconnecter
- Récupérer son accès en cas d'oubli de mot de passe
- Modifier son mot de passe
- Consulter et modifier son profil

Le système devra impérativement bloquer l'accès à toutes les pages protégées pour tout utilisateur non authentifié.

6.2 Module Utilisateurs

Réservé à l'administrateur, ce module permettra de :

- Créer un nouvel utilisateur
- Modifier les informations d'un utilisateur existant
- Désactiver un utilisateur
- Consulter la liste complète des utilisateurs
- Attribuer ou modifier le rôle d'un utilisateur
- Rechercher un utilisateur

Chaque utilisateur sera défini par les attributs suivants : Nom, Prénom, Email, Téléphone, Mot de passe, Rôle, Statut, Date de création.

6.3 Module Clients

Ce module permettra de gérer le portefeuille clients de l'entreprise :

- Créer un nouveau client
- Modifier les informations d'un client
- Désactiver ou supprimer un client
- Consulter la fiche d'un client
- Rechercher un client
- Consulter l'historique des commandes d'un client

Chaque client sera défini par : ID, Nom, Prénom, Email, Téléphone, Adresse, Date de création, Statut.

6.4 Module Produits et Services

Ce module permettra de gérer le catalogue de l'entreprise :

- Ajouter un produit ou un service
- Modifier ses informations
- Désactiver un produit
- Consulter la liste avec filtrage par catégorie
- Rechercher un produit
- Gérer les alertes de stock

Chaque produit sera défini par : ID, Nom, Description, Catégorie, Prix, Quantité, Seuil d'alerte, Statut, Date de création.

6.5 Module Commandes

Une commande est toujours associée à un client. Ce module permettra de :

- Créer une nouvelle commande
- Ajouter plusieurs produits ou services à une commande
- Modifier une commande existante
- Changer le statut d'une commande
- Annuler une commande

Cycle de vie d'une commande :

    - Brouillon → Confirmée → En préparation → Terminée

Chaque commande contiendra : Numéro, Client, Date, Articles, Sous-total, Réduction, Total, Statut, Utilisateur ayant créé la commande.

6.6 Module Paiements

Ce module permettra d'enregistrer et de suivre les paiements liés aux commandes :

- Enregistrer un paiement
- Associer un paiement à une commande
- Consulter le statut de paiement d'une commande

Modes de paiement supportés : espèces, mobile money, virement bancaire, carte bancaire.

Le système déterminera automatiquement si une commande est : non payée, partiellement payée ou entièrement payée.

6.7 Tableau de bord

Après connexion, l'utilisateur verra un tableau de bord adapté à son rôle. Il affichera notamment :

- Nombre total de clients
- Nombre total de commandes
- Chiffre d'affaires total
- Nombre de commandes en cours
- Nombre de paiements en attente
- Évolution des ventes sur une période donnée
- Commandes récentes
- Meilleurs produits
- Clients et paiements récents

6.8 Historique et journal d'activité

Le système enregistrera automatiquement les actions importantes des utilisateurs afin d'assurer la traçabilité des opérations. Exemples d'entrées journalisées :

- Création d'un client
- Création ou modification d'une commande
- Enregistrement d'un paiement
- Désactivation d'un utilisateur

6.9 Recherche et filtrage

Tous les modules principaux proposeront les fonctionnalités suivantes :

- Recherche par mot-clé
- Filtrage par critères
- Tri par colonnes
- Pagination des résultats

7. ARCHITECTURE TECHNIQUE

7.1 Stack technologique

Couche	Technologie
Langage principal	TypeScript
Framework frontend	React
Framework full-stack	Next.js
CSS / Design	TailwindCSS
Base de données	PostgreSQL
ORM	Prisma
Authentification	NextAuth.js (JWT)
Tests	Jest / Testing Library
Conteneurisation	Docker / Docker Compose
Déploiement	Vercel (frontend) · Railway (BDD)
Versionnement	Git / GitHub

7.2 Architecture générale

Le projet adopte une architecture full-stack unifiée grâce à Next.js, ce qui permet de gérer à la fois le frontend et le backend (API Routes) dans le même projet, sans avoir à apprendre un second framework backend.

Flux technique : TypeScript → React → Next.js → API Routes → Prisma → PostgreSQL

8. MODÉLISATION DE LA BASE DE DONNÉES

La base de données PostgreSQL comportera les tables principales suivantes :

- users : Utilisateurs du système
- roles : Rôles et niveaux d'accès

- permissions : Permissions associées aux rôles
- clients : Clients de l'entreprise
- categories : Catégories de produits
- products : Produits et services
- orders : Commandes
- order_items : Lignes de commande
- payments : Paiements
- activity_logs : Journal d'activité

Relations principales :

- Un rôle peut être attribué à plusieurs utilisateurs
- Un utilisateur peut créer plusieurs commandes et journaux d'activité
- Un client peut avoir plusieurs commandes
- Une commande peut contenir plusieurs produits via order_items
- Une commande peut être associée à plusieurs paiements

Note importante : la modélisation finale sera réalisée et justifiée par le stagiaire pendant la phase de conception. L'objectif est de comprendre pourquoi chaque table existe et quelles contraintes s'appliquent.

9. API REST

L'application communiquera avec le serveur via une API REST. Les principaux endpoints seront :

Méthode	Endpoint	Description
GET	/api/clients	Liste de tous les clients
GET	/api/clients/:id	Détails d'un client
POST	/api/clients	Créer un client
PUT	/api/clients/:id	Modifier un client
DELETE	/api/clients/:id	Supprimer un client
GET	/api/products	Liste des produits
POST	/api/products	Créer un produit
GET	/api/orders	Liste des commandes
POST	/api/orders	Créer une commande
GET	/api/payments	Liste des paiements
POST	/api/payments	Enregistrer un paiement
GET	/api/users	Liste des utilisateurs
POST	/api/auth/login	Connexion
POST	/api/auth/logout	Déconnexion

Chaque endpoint devra gérer : les codes de statut HTTP appropriés, la validation des données entrantes, la gestion des erreurs, l'authentification et l'autorisation.

10. SÉCURITÉ

Le projet devra intégrer les bonnes pratiques de sécurité suivantes :

- Hachage des mots de passe avec bcrypt
- Gestion des sessions via JWT (JSON Web Tokens)
- Protection des routes privées
- Validation des données côté client et côté serveur
- Utilisation de variables d'environnement pour les données sensibles
- Contrôle d'accès basé sur les rôles (RBAC)
- Protection contre les injections SQL via Prisma ORM
- Limitation des informations exposées par l'API

L'objectif n'est pas de construire un système bancaire, mais d'acquérir de bonnes habitudes de développement sécurisé dès le début.

11. TESTS

Tests unitaires

Tester une fonction ou un composant de manière isolée et indépendante.

Tests d'intégration

Tester l'interaction entre plusieurs composants ou modules du système.

Tests fonctionnels

Vérifier des scénarios complets, par exemple :

- Connexion → Création d'un client → Création d'une commande → Enregistrement d'un paiement → Consultation du tableau de bord

12. VERSIONNEMENT AVEC GIT

L'ensemble du projet sera versionné avec Git. Les commits devront être clairs, descriptifs et réguliers. Exemples de messages de commit :

- feat: add client creation form
- feat: add order management module
- fix: correct order total calculation
- refactor: improve authentication service
- docs: update API documentation

13. CONTENEURISATION AVEC DOCKER

À la fin du projet, l'application sera conteneurisée avec Docker. Les éléments suivants seront appris et maîtrisés :

- Rédaction d'un Dockerfile pour l'application Next.js
- Configuration de Docker Compose pour orchestrer les services
- Gestion des variables d'environnement dans les conteneurs
- Utilisation des volumes pour la persistance des données
- Configuration des réseaux Docker

Architecture Docker cible : un conteneur Next.js connecté à un conteneur PostgreSQL via Docker Compose.

14. DÉPLOIEMENT

La dernière étape consistera à rendre l'application accessible en ligne. Le stagiaire devra maîtriser :

- Le cycle complet : Code → Git → Build → Déploiement → Application en ligne
- La distinction entre environnement de développement et environnement de production
- La gestion des variables d'environnement en production
- La lecture des logs de production
- La configuration d'un domaine et du protocole HTTPS